

# Agora

## Документация

---

**Екип: Zlaten Vek**

React • NestJS • PostgreSQL • OpenAI • Docker • Kubernetes

Екип: Zlaten Vek

Технологичен стек: React + Vite + TypeScript (web), NestJS + TypeScript (api), PostgreSQL (Supabase) + Prisma, OpenAI, Docker, Kubernetes (k3s), Traefik, Prometheus + Grafana + Alertmanager, GitHub Actions, ArgoCD (GitOps).

# Съдържание

|                                                      |    |
|------------------------------------------------------|----|
| Съдържание                                           | 2  |
| Увод                                                 | 5  |
| 1. Анализ и проучване                                | 5  |
| 1.1 Предметна област и целева аудитория              | 5  |
| 1.2 Преглед на съществуващи решения                  | 6  |
| 1.2.1 Официални обществени консултации (strategy.bg) | 6  |
| 1.2.2 ChatGPT / Claude (директно)                    | 6  |
| 1.2.3 Академични дебатни инструменти                 | 6  |
| Обобщение - сравнителна матрица                      | 6  |
| 1.3 Аргументация за избор на технологии              | 7  |
| 2. Проектиране                                       | 8  |
| 2.1 Функционални изисквания                          | 8  |
| 2.2 Архитектура на системата                         | 9  |
| 2.3 Инфраструктурна диаграма                         | 11 |
| 2.4 Схема на базата данни                            | 12 |
| Таблици                                              | 12 |
| Релации                                              | 15 |
| 2.5 UML class диаграма                               | 15 |
| 3. Реализация                                        | 16 |
| 3.1 Файлова структура                                | 17 |
| 3.2 Сървърна част - API                              | 17 |
| Жизнен цикъл на дебата                               | 17 |
| REST endpoint-и                                      | 17 |

|                                                                       |           |
|-----------------------------------------------------------------------|-----------|
| Класова йерархия на агентите (Strategy + наследяване) . . . . .       | 18        |
| AgentOrchestrator (Chain of Responsibility + конкурентност) . . . . . | 19        |
| LLM интеграция . . . . .                                              | 19        |
| Обработка на PDF . . . . .                                            | 19        |
| Custom exceptions . . . . .                                           | 19        |
| <b>3.3 Клиентска част - React . . . . .</b>                           | <b>20</b> |
| Маршрути . . . . .                                                    | 20        |
| Feature модули . . . . .                                              | 20        |
| SSE стрийминг . . . . .                                               | 20        |
| Двата режима . . . . .                                                | 21        |
| TanStack Query . . . . .                                              | 21        |
| Основни компоненти . . . . .                                          | 21        |
| <b>3.4 База данни - модели и заявки . . . . .</b>                     | <b>21</b> |
| <b>3.5 Тестване . . . . .</b>                                         | <b>21</b> |
| <b>4. Инфраструктура . . . . .</b>                                    | <b>22</b> |
| 4.1 Docker конфигурация . . . . .                                     | 22        |
| 4.2 CI/CD Pipeline . . . . .                                          | 22        |
| 4.3 Kubernetes и наблюдаемост . . . . .                               | 23        |
| 4.4 Инструкции за стартиране . . . . .                                | 24        |
| <b>5. Екранни снимки . . . . .</b>                                    | <b>24</b> |
| Табло (Dashboard) . . . . .                                           | 24        |
| Стая на дебата (Stage view) . . . . .                                 | 25        |
| Синтез (заключение) . . . . .                                         | 27        |
| Профил . . . . .                                                      | 28        |
| <b>6. AI инструменти . . . . .</b>                                    | <b>28</b> |
| 6.1 Claude Code . . . . .                                             | 28        |
| 6.2 GitHub Copilot . . . . .                                          | 28        |
| <b>Заклучение . . . . .</b>                                           | <b>28</b> |

|                      |    |
|----------------------|----|
| Источници . . . . .  | 29 |
| Приложения . . . . . | 29 |

# Увод

Гражданите рядко имат реален поглед върху това как един законопроект ще се отрази на различните обществени групи. Официалните обществени консултации са бавни, формални и недостъпни за обикновения човек, а директното задаване на въпрос към AI чатбот дава една единствена, осреднена гледна точка - без сблъсък на интереси, без trade-off-и, без структуриран дебат.

Agora е уеб приложение, което симулира структуриран многостранен дебат върху законопроекти и обществени политики. Потребителят качва PDF с текста на законопроект или го въвежда директно. Системата автоматично анализира съдържанието, определя засегнатите обществени групи и генерира по един AI агент за всяка група с конкретна персона - демография, интереси, страхове, приоритети. Агентите дебатира в структурирани рундове, видими като жив чат с аватари и цветове. Финален неутрален агент-съдия синтезира дискусията и извежда основните противоречия, общите точки и възможни компромиси.

Целта не е да се даде "правилният" отговор. Целта е да се покажат реалните притеснения и trade-off-ите от различните гледни точки на обществото - нещо, което нито една от съществуващите алтернативи не прави автоматично.

## 1. Анализ и проучване

### 1.1 Предметна област и целева аудитория

Предметната област е гражданско участие в законодателния процес - конкретно осмислянето на това как предложена политика засяга различни обществени групи. Един жилищен закон например не засяга "обществото" еднообразно: наемателят, наемодателят, строителната компания, общинският съветник и младият човек, търсещ първи дом, имат коренно различни интереси и страхове. Тези гледни точки рядко са събрани на едно място в разбираем вид.

Целева аудитория:

| Група                                         | Нужда                                                   | Болка днес                                               |
|-----------------------------------------------|---------------------------------------------------------|----------------------------------------------------------|
| Граждани, засегнати от конкретна политика     | Бързо да разберат как ги засяга даден закон             | Юридическият текст е недостъпен; нямат време да го четат |
| Журналисти и анализатори                      | Бърз преглед на конфликтните точки и засегнатите страни | Ръчният анализ е бавен                                   |
| Студенти и преподаватели (право, политология) | Учебен инструмент за разбиране на trade-off-и           | Липсва интерактивен симулатор                            |
| НПО и граждански организации                  | Изходна точка за становище                              | Започват от нула при всеки нов законопроект              |

Таблица 1: Целева аудитория

**Ключови нужди:** разбираемо обобщение на законопроекта; идентификация на засегнатите групи; структуриран сблъсък на позициите; неутрален синтез с конкретни компромисни точки.

## 1.2 Преглед на съществуващи решения

### 1.2.1 Официални обществени консултации (strategy.bg)

Официалният канал за подаване на становища по нормативни актове.

| Аспект   | Оценка                                                                                            |
|----------|---------------------------------------------------------------------------------------------------|
| Предлага | Формален, легитимен канал за становища                                                            |
| Липсва   | Бавно, сложно, недостъпно за обикновения гражданин; не симулира дебат, само събира писмени мнения |

Таблица 2: Официални обществени консултации

### 1.2.2 ChatGPT / Claude (директно)

Общодостъпни LLM асистенти.

| Аспект   | Оценка                                                                                                                                                           |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Предлага | Може да анализира текст и да даде обобщение                                                                                                                      |
| Липсва   | Единична, осреднена гледна точка; няма структуриран дебат между множество персони; няма автоматично определяне на засегнатите групи; няма синтез на противоречия |

Таблица 3: ChatGPT / Claude (директно)

### 1.2.3 Академични дебатни инструменти

Структури за формален дебат (Karl Popper, British Parliamentary).

| Аспект   | Оценка                                                                                  |
|----------|-----------------------------------------------------------------------------------------|
| Предлага | Структурирани аргументи и правила за разисквания                                        |
| Липсва   | Не са автоматизирани; изискват ръчно въвеждане на позиции и хора, които да ги защитават |

Таблица 4: Академични дебатни инструменти

### Обобщение - сравнителна матрица

| Възможност                          | Обществени консултации | ChatGPT/Claude | Академични дебати | Agora |
|-------------------------------------|------------------------|----------------|-------------------|-------|
| Автоматичен анализ на текста        | Не                     | Да             | Не                | Да    |
| Авто-определяне на засегнати групи  | Не                     | частично       | Не                | Да    |
| Множество гледни точки едновременно | частично               | Не             | Да                | Да    |

| Възможност                      | Обществени консултации | ChatGPT/Claude | Академични дебати | Agora |
|---------------------------------|------------------------|----------------|-------------------|-------|
| Структуриран дебат в рундове    | Не                     | Не             | Да                | Да    |
| Неутрален синтез + компромиси   | Не                     | частично       | Не                | Да    |
| Достъпно за обикновен гражданин | Не                     | Да             | Не                | Да    |

Таблица 5: Сравнителна матрица на решенията

Agora е новото решение, което покрива всички горни пропуски: автоматичен анализ + динамични агенти + структуриран дебат + синтез.

### 1.3 Аргументация за избор на технологии

| Технология                | Защо                                                                                                                                                                                                     |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| React + Vite + TypeScript | Бърз dev сървър (HMR), типова безопасност споделена с бекенда, голяма екосистема. Чатът с реалновремеви стрийминг изисква отзивчив компонентен модел.                                                    |
| NestJS + TypeScript       | Модулна архитектура с вграден Dependency Injection - позволява чиста слоеста архитектура. Споделя типове и DTO-та с фронтенда през @agora/shared.                                                        |
| PostgreSQL (Supabase)     | Данните имат ясни релации (debate → personas → messages), нужни са наредени заявки по round_number и транзакционна консистентност при едновременен стрийминг. Supabase добавя готова Auth (email + JWT). |
| Prisma ORM                | Типово-безопасен достъп до БД, декларативни миграции с timestamp, eager/lazy loading контрол.                                                                                                            |
| OpenAI                    | Стрийминг token-by-token, ниска цена за дълги дебати с много агенти, добро следване на JSON схеми за структурирани изходи (анализ, синтез).                                                              |
| Server-Sent Events (SSE)  | Еднопосочен стрийминг сървър → клиент пасва идеално за token-by-token реплики; по-лек от WebSocket за случая.                                                                                            |
| Docker + k3s + Traefik    | Лек Kubernetes за самостоятелен хостинг; декларативни манифести (IaC); HPA за автоскалиране на API и web.                                                                                                |
| pnpm + Turborepo          | Монорепо с workspace кеширане - бекенд, фронтенд и споделени типове в едно репо.                                                                                                                         |

Таблица 6: Избор на технологии

**Защо релационна, а не NoSQL:** релациите са присъщи на домейна (дебат съдържа персони, персони създават съобщения, рундове групират съобщения). Нужни са наредени заявки по round\_number и транзакционна консистентност при каскадно изтриване и едновременен стрийминг. MongoDB не би дал предимство тук.

## 2. Проектиране

### 2.1 Функционални изисквания

#### FR-1: Аутентикация

| ID     | Изискване                                                                    | Приоритет |
|--------|------------------------------------------------------------------------------|-----------|
| FR-1.1 | Регистрация с email и парола (Supabase Auth)                                 | Висок     |
| FR-1.2 | Потвърждение на email преди вход                                             | Висок     |
| FR-1.3 | Вход с email и парола; издаване на JWT                                       | Висок     |
| FR-1.4 | Защитени маршрути - неавтентикиран потребител се пренасочва към /login       | Висок     |
| FR-1.5 | Преглед и редакция на профил (име); изтриване на акаунт с каскадно изтриване | Среден    |

Таблица 7: Функционални изисквания - Аутентикация

#### FR-2: Създаване на дебат

| ID     | Изискване                                                                    | Приоритет |
|--------|------------------------------------------------------------------------------|-----------|
| FR-2.1 | Качване на PDF (drag-drop или файлов избор) с текста на законопроект         | Висок     |
| FR-2.2 | Алтернативно директно въвеждане на текст (минимум 200 символа)               | Висок     |
| FR-2.3 | Задължително заглавие на законопроекта (макс. 200 символа)                   | Среден    |
| FR-2.4 | Валидация на PDF - magic bytes, отхвърляне на сканирани PDF без текстов слой | Висок     |
| FR-2.5 | Ограничение на дължината (200 - 200 000 символа)                             | Среден    |

Таблица 8: Функционални изисквания - Създаване на дебат

#### FR-3: Анализ и персони

| ID     | Изискване                                                                    | Приоритет |
|--------|------------------------------------------------------------------------------|-----------|
| FR-3.1 | AnalysisAgent извежда засегнати групи, ключови промени и спорни точки        | Висок     |
| FR-3.2 | По една Persona за всяка засегната група (4-6 групи)                         | Висок     |
| FR-3.3 | Преглед на персоните преди старт - картичка с демография, интереси, страхове | Висок     |
| FR-3.4 | Премахване на персона (но не под 2 персони)                                  | Среден    |
| FR-3.5 | Опресняване на статуса на анализа на всеки 2 секунди до завършване           | Нисък     |

Таблица 9: Функционални изисквания - Анализ и персони

#### FR-4: Дебат



| ID     | Изискване                                                                      | Приоритет |
|--------|--------------------------------------------------------------------------------|-----------|
| FR-4.1 | Структурирани рундове: Позиция → Контрааргументи → Общо основание              | Висок     |
| FR-4.2 | Всеки агент получава пълната история на дебата като контекст                   | Висок     |
| FR-4.3 | Token-by-token стрийминг на репликите по SSE                                   | Висок     |
| FR-4.4 | Режим auto-play - дискусията тече до край автоматично                          | Висок     |
| FR-4.5 | Режим стъпка-по-стъпка - потребителят сам преминава към следващия рунд         | Висок     |
| FR-4.6 | Класификация на емоция за всяка реплика (calm/confident/pensive/anxious/tense) | Нисък     |

Таблица 10: Функционални изисквания - Дебат

### FR-5: Синтез (заключение)

| ID     | Изискване                                                                  | Приоритет |
|--------|----------------------------------------------------------------------------|-----------|
| FR-5.1 | JudgeAgent извежда противоречия, общи точки и компромисни предложения      | Висок     |
| FR-5.2 | За всяка персона - как се е променила позицията ѝ в хода на дебата (shift) | Висок     |
| FR-5.3 | Заключително изявление на съдията                                          | Среден    |
| FR-5.4 | Регенериране на синтеза                                                    | Среден    |
| FR-5.5 | Експорт на синтеза в PDF                                                   | Нисък     |

Таблица 11: Функционални изисквания - Синтез

### FR-6: Табло (dashboard)

| ID     | Изискване                                                         | Приоритет |
|--------|-------------------------------------------------------------------|-----------|
| FR-6.1 | Картички с всички дебати (заглавие, код, дата, участници, статус) | Висок     |
| FR-6.2 | Филтри: всички / активни / синтезирани / чернови                  | Среден    |
| FR-6.3 | Обобщена статистика (брой дебати, участници, последна активност)  | Среден    |
| FR-6.4 | Панел за активни в момента дебати                                 | Нисък     |

Таблица 12: Функционални изисквания - Табло

## 2.2 Архитектура на системата

Системата следва Layered Architecture и от двете страни. Бизнес логиката е изцяло отделена от транспорта (HTTP/SSE) и от данните (SQL).

**API (вътрешна посока):** presentation → application → domain, а infrastructure имплементира port-овете на domain и се закача в module файла. domain не зависи от нищо - няма NestJS, няма Prisma, няма HTTP типове.

| Слой           | Отговорност                                | Пример                                                              |
|----------------|--------------------------------------------|---------------------------------------------------------------------|
| Presentation   | HTTP контролери, SSE endpoint-и, валидация | DebateController, JudgeController                                   |
| Application    | Use-case оркестрация, услуги               | AgentOrchestrator, DebateService, AnalysisService                   |
| Domain         | Чисти entity-та, port-ове, бизнес правила  | BaseAgent, IDebateAgent, IDebateRepository                          |
| Infrastructure | Имплементации на port-овете                | PrismaDebateRepository, OpenAIStreamingClient, PdfBillTextExtractor |

Таблица 13: Слоевете на API архитектурата

**Web (надолу):** app → pages → features → entities → shared. По-нисък слой никога не импортира от по-висок.

**NestJS модули** (apps/api/src/modules/): auth, user, debate, persona, agent, analysis, judge, round, metrics, health, prisma. Всеки feature модул има свои domain / application / infrastructure / presentation папки.

**Design patterns (с обосновка):**

| Pattern                 | Къде                                                                                                               | Защо                                                                             |
|-------------------------|--------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Strategy                | IDebateAgent.generateResponse(context) - всеки агент (PersonaAgent, JudgeAgent, AnalysisAgent) е отделна стратегия | Различни персони, единна точка на извикване; нов тип агент не променя извикващия |
| Chain of Responsibility | AgentOrchestrator подава хода последователно на всеки агент в реда на рунда                                        | Подреден обход на агентите без те да знаят един за друг                          |
| Factory                 | PersonaAgentFactory.create(persona) и .createJudge() строят агенти от AnalysisResult                               | Капсулира конструирането и инжектирането на ILLMClient                           |
| Repository              | Всички I*Repository port-ове + Prisma*Repository имплементации                                                     | Абстрахира достъпа до БД зад domain интерфейс                                    |
| Dependency Injection    | NestJS DI - услугите зависят от symbol-token-и (LLM_CLIENT, DEBATE_REPOSITORY)                                     | Размяна на имплементация без промяна на потребителя                              |

Таблица 14: Design patterns

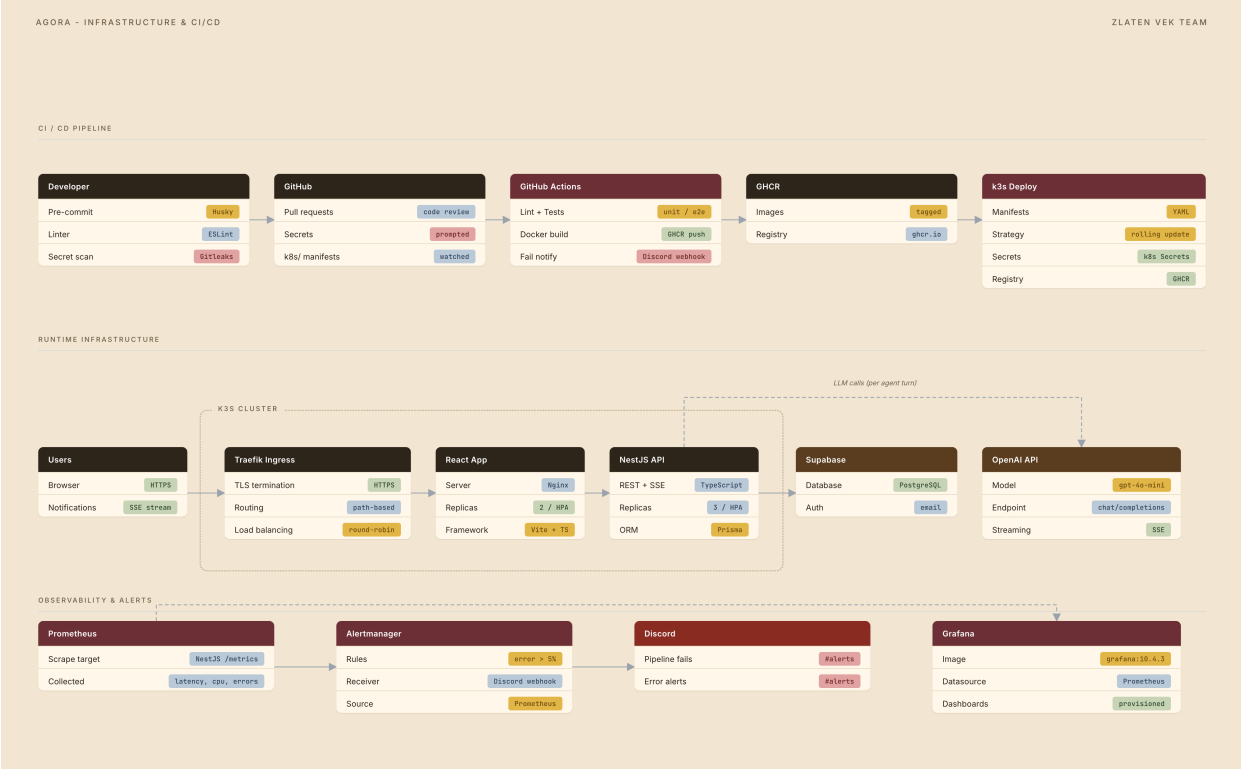
**SOLID принципи:**

| Принцип | Приложение                                                                                                                                |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------|
| SRP     | AgentOrchestrator само управлява реда на рундовете и агентите; не генерира съдържание и не пише в БД директно (през repository port-ове). |
| OCP     | Нов тип агент се добавя чрез нов подклас на BaseAgent, без промяна на AgentOrchestrator.                                                  |

| Принцип | Приложение                                                                                                        |
|---------|-------------------------------------------------------------------------------------------------------------------|
| DIP     | AgentOrchestrator зависи от IDebateAgent, не от конкретни агенти; услугите зависят от I*Repository, не от Prisma. |

Таблица 15: SOLID принципи

2.3 Инфраструктурна диаграма



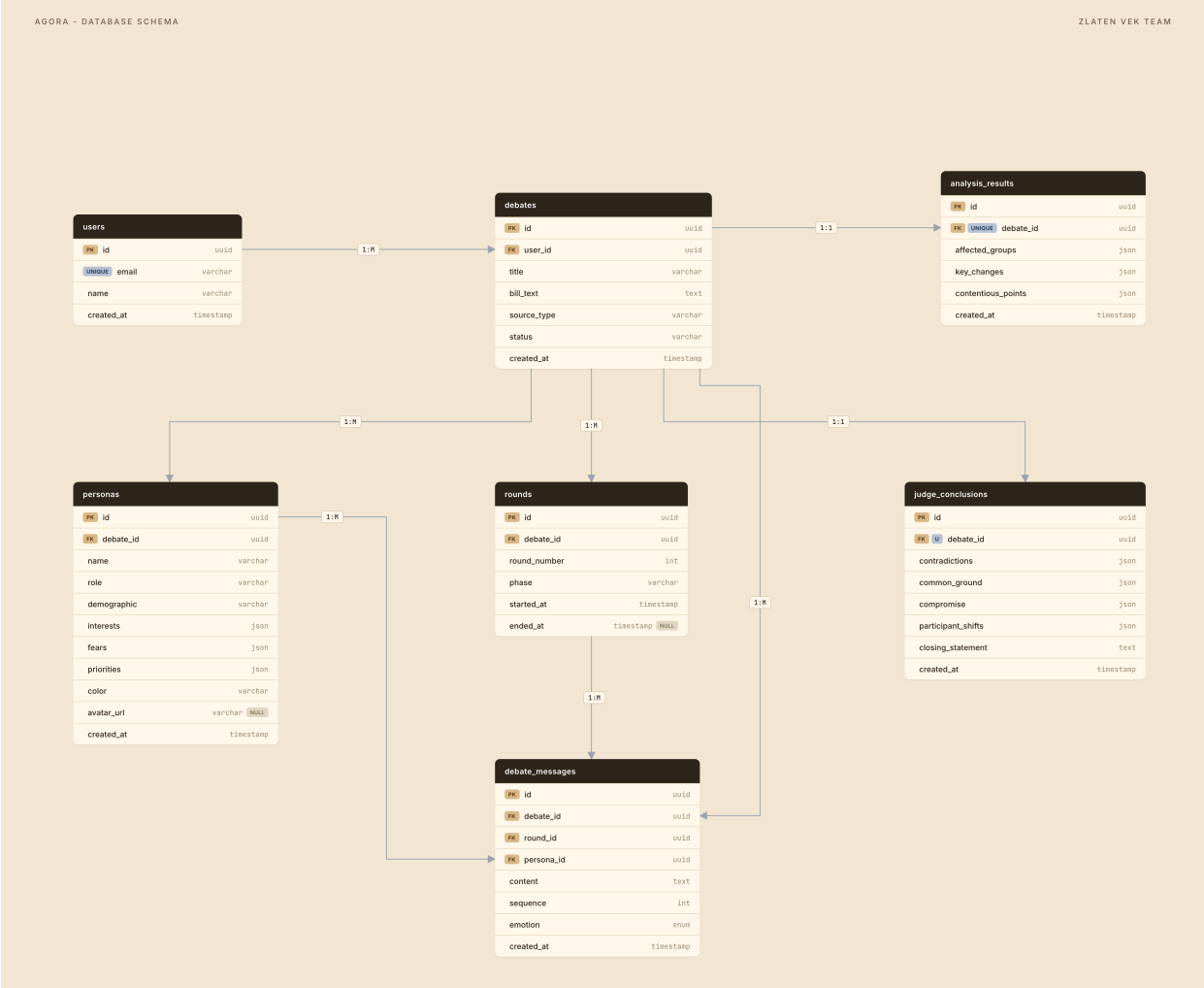
Фигура 1: Инфраструктурна диаграма

| Компонент                           | Роля                                                                                                      |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Traefik IngressRoute                | Маршрутизация: PathPrefix (/api) → api service (priority 100), PathPrefix (/) → web service (priority 10) |
| Web Deployment                      | 2 реплики, serve сервера Vite билда, port 80                                                              |
| API Deployment                      | 3 реплики, NestJS, port 3000, initContainer пуска Prisma миграции, non-root user 1001                     |
| API HPA                             | Min 3 / Max 6 реплики, scale при CPU > 70%                                                                |
| Web HPA                             | Min 2 / Max 4 реплики, scale при CPU > 70%                                                                |
| Supabase (PostgreSQL)               | Управлявана база + Auth                                                                                   |
| OpenAI API                          | LLM генерация, стрийминг                                                                                  |
| Prometheus + Grafana + Alertmanager | Метрики, дашборди, алерти → Discord                                                                       |
| GHCR                                | Container registry за image-ите                                                                           |

| Компонент | Роля                                                                                                      |
|-----------|-----------------------------------------------------------------------------------------------------------|
| ArgoCD    | GitOps контролер: следи k8s/ на main и автоматично синхронизира манифестите в клъстера (prune + selfHeal) |

Таблица 16: Компоненти на инфраструктурата

2.4 Схема на базата данни



Фигура 2: ER диаграма на базата данни

Базата е в 3NF.

Таблицы

users - системни потребители.

| Колона | Тип  | Бележка |
|--------|------|---------|
| id     | uuid | PK      |
| email  | text | UNIQUE  |
| name   | text |         |

| Колона     | Тип       | Бележка       |
|------------|-----------|---------------|
| created_at | timestamp | default now() |

Таблица 17: Таблица users

debates - една дебатна сесия върху законопроект.

| Колона      | Тип       | Бележка                      |
|-------------|-----------|------------------------------|
| id          | uuid      | PK                           |
| user_id     | uuid      | FK → users.id                |
| title       | text      | заглавие                     |
| bill_text   | text      | пълен текст на законопроекта |
| source_type | text      | "pdf" или "text"             |
| status      | text      | default "draft"              |
| created_at  | timestamp | default now()                |

Таблица 18: Таблица debates

personas - една засегната група (отделна таблица, не се дублира в съобщенията).

| Колона      | Тип       | Бележка                                   |
|-------------|-----------|-------------------------------------------|
| id          | uuid      | PK                                        |
| debate_id   | uuid      | FK → debates.id                           |
| name        | text      | име на персоната                          |
| role        | text      | роля/група (напр. "наемател"), default "" |
| demographic | text      | демография                                |
| interests   | jsonb     | масив от интереси                         |
| fears       | jsonb     | масив от страхове                         |
| priorities  | jsonb     | масив от приоритети                       |
| color       | text      | UI цвят                                   |
| avatar_url  | text?     | опционален аватар                         |
| created_at  | timestamp | default now()                             |

Таблица 19: Таблица personas

rounds - един рунд от дебата.

| Колона | Тип  | Бележка |
|--------|------|---------|
| id     | uuid | PK      |

| Колона       | Тип        | Бележка                           |
|--------------|------------|-----------------------------------|
| debate_id    | uuid       | FK → debates.id                   |
| round_number | int        | пореден номер (1,2,3)             |
| phase        | text       | Position / Counter / CommonGround |
| started_at   | timestamp  | default now()                     |
| ended_at     | timestamp? | при завършване                    |

Таблица 20: Таблица rounds

Индекс: (debate\_id, round\_number).

debate\_messages - една реплика на една персона в един рунд.

| Колона     | Тип          | Бележка             |
|------------|--------------|---------------------|
| id         | uuid         | PK                  |
| debate_id  | uuid         | FK → debates.id     |
| round_id   | uuid         | FK → rounds.id      |
| persona_id | uuid         | FK → personas.id    |
| content    | text         | текст на репликата  |
| sequence   | int          | ред на хода в рунда |
| emotion    | enum Emotion | default calm        |
| created_at | timestamp    | default now()       |

Таблица 21: Таблица debate\_messages

Индекси: (debate\_id, created\_at) за хронология, (round\_id, sequence) за реда на ходовете.

analysis\_results - резултат от началния анализ (1:1 с debate).

| Колона             | Тип       | Бележка                |
|--------------------|-----------|------------------------|
| id                 | uuid      | PK                     |
| debate_id          | uuid      | FK UNIQUE → debates.id |
| affected_groups    | jsonb     | засегнати групи        |
| key_changes        | jsonb     | ключови промени        |
| contentious_points | jsonb     | спорни точки           |
| created_at         | timestamp | default now()          |

Таблица 22: Таблица analysis\_results

judge\_conclusions - финален синтез (1:1 с debate).

| Колона             | Тип       | Бележка                |
|--------------------|-----------|------------------------|
| id                 | uuid      | PK                     |
| debate_id          | uuid      | FK UNIQUE → debates.id |
| contradictions     | jsonb     | противоречия           |
| common_ground      | jsonb     | обща точки             |
| compromise         | jsonb     | компромиси             |
| participant_shifts | jsonb     | промени на позициите   |
| closing_statement  | text      | заключително изявление |
| created_at         | timestamp | default now()          |

Таблица 23: Таблица judge\_conclusions

**Enum** Emotion: calm, confident, pensive, anxious, tense.

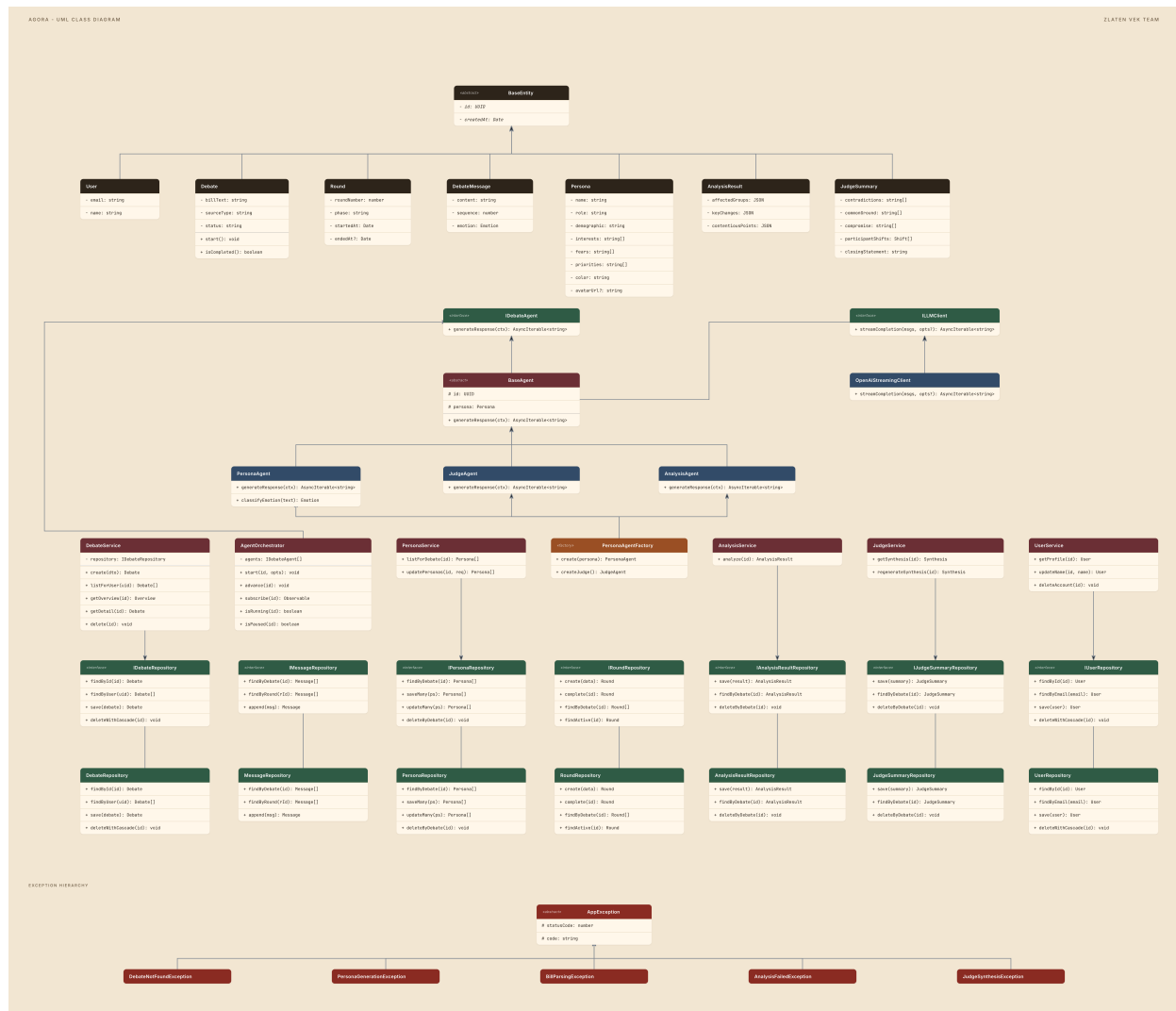
### Релации

- users 1:N debates
- debates 1:N personas
- debates 1:N rounds
- debates 1:N debate\_messages
- personas 1:N debate\_messages
- rounds 1:N debate\_messages
- debates 1:1 analysis\_results (опц.)
- debates 1:1 judge\_conclusions (опц.)

**Нормализация (3NF):** personas е отделна таблица - демографските данни не се дублират в debate\_messages. round\_number е атрибут на рунда (int), не конкатениран низ. Всички полета са атомарни или FK; няма изводими дублирани полета. Каскадното изтриване се прави с явни \$transaction() в repository-тата (на ниво схема всички FK са ON DELETE RESTRICT).

**ORM (Prisma):** debate\_messages се **eager-load**-ват заедно с persona (include: { persona: true, round: { select: { roundNumber } } }) - чат UI винаги има нужда от персоната. Списъкът с персони и пълната история се зареждат **lazy** при нужда.

## 2.5 UML class диаграма



Фигура 3: UML class диаграма

**Наследяване:** BaseAgent → PersonaAgent, JudgeAgent, AnalysisAgent. BaseAgent имплементира абстракцията IDebateAgent (id + generateResponse(context): AsyncIterable<string>).

**Абстракция:** IDebateAgent - единен интерфейс за всички агенти (Strategy).

**Полиморфизъм:** AgentOrchestrator вика generateResponse(context) върху всеки агент, без да знае конкретния му тип.

**Капсулация:** историята на дебата и състоянието на сесията се управляват вътрешно в DebateSession / orchestrator-a; persona полетата (interests, fears, priorities) са капсулирани в PersonaEntity.

**Композиция:** Debate съдържа Round-ове, Round съдържа DebateMessage-и.

**Generics и Collections:** Map<string, DebateSession> за активни сесии, Promise<DebateEntity | null> (Optional-еквивалент) при търсене по ID, List<DebateMessage> за историята.

## 3. Реализация



### 3.1 Файлова структура



Листинг 1: Файлова структура на проекта

### 3.2 Сървърна част - API

#### Жизнен цикъл на дебата

1. **Създаване** - `DebateService.create()` записва чернова и пуска анализа detached (асинхронно).
2. **Анализ** - `AnalysisService.analyze()` пуска `AnalysisAgent` (с 2 опита `retry`), парсва JSON изхода и създава персони. Статусът минава през `Analyzing` → `PersonasPending`.
3. **Старт** - `AgentOrchestrator.start()` създава `DebateSession`, строи агентите през `PersonaAgentFactory`, пуска `runSession()`.
4. **Рундове** - `runSession()` обхожда `ROUND_PHASES = [Position, Counter, CommonGround]`; за всеки агент стриймва токени, класифицира емоция, записва съобщение.
5. **Синтез** - след последния рунд `JudgeAgent` синтезира; `saveJudgeSummary()` парсва и валидира JSON-а.

#### REST endpoint-и

| Метод | Път      | Описание                                         |
|-------|----------|--------------------------------------------------|
| POST  | /debates | Създаване (multipart: billTitle + файл/billText) |

| Метод  | Път                               | Описание                                  |
|--------|-----------------------------------|-------------------------------------------|
| GET    | /debates                          | Списък дебати на потребителя (пагинация)  |
| GET    | /debates/me/chamber               | Обобщена статистика                       |
| POST   | /debates/:id/start                | Старт (?mode=step за стъпка-по-стъпка)    |
| GET    | /debates/:id/stream               | <b>SSE</b> поток (text/event-stream)      |
| POST   | /debates/:id/advance              | Преминаване към следващ рунд (step режим) |
| GET    | /debates/:id/overview             | Обзор (ключови промени, персони, прогрес) |
| GET    | /debates/:id                      | Пълни детайли с история                   |
| DELETE | /debates/:id                      | Каскадно изтриване                        |
| POST   | /debates/:debateId/analysis/retry | Повторен анализ                           |
| GET    | /debates/:debateId/personas       | Списък персони                            |
| PATCH  | /debates/:debateId/personas       | Bulk добавяне/промяна/премахване          |
| GET    | /debates/:id/synthesis            | Синтез на съдията                         |
| POST   | /debates/:id/synthesis/regenerate | Регенериране на синтеза                   |
| GET    | /auth/me, /users/me               | Профил                                    |
| PATCH  | /users/me                         | Промяна на име                            |
| DELETE | /users/me                         | Изтриване на акаунт                       |
| GET    | /health                           | Health check (публичен)                   |
| GET    | /metrics                          | Prometheus метрики (публичен)             |

Таблица 24: REST endpoint-и

### Класова йерархия на агентите (Strategy + наследяване)

| Клас         | Файл                           | Роля                                                                                                  |
|--------------|--------------------------------|-------------------------------------------------------------------------------------------------------|
| IDebateAgent | agent/domain/i-debate-agent.ts | Интерфейс: id, generateResponse(context)                                                              |
| BaseAgent    | agent/domain/base-agent.ts     | Абстрактен база - id, persona, llmClient                                                              |
| PersonaAgent | agent/domain/persona-agent.ts  | Дебатира като персона; ROUND_INSTRUCTIONS (Position/Counter/CommonGround); classifyEmotion()          |
| JudgeAgent   | agent/domain/judge-agent.ts    | Синтезира - JSON изход: contradictions, commonGround, compromise, participantShifts, closingStatement |

| Клас                | Файл                                  | Роля                                                    |
|---------------------|---------------------------------------|---------------------------------------------------------|
| AnalysisAgent       | analysis/domain/analysis-agent.ts     | JSON изход: groups (4-6), keyChanges, contentiousPoints |
| PersonaAgentFactory | agent/domain/persona-agent-factory.ts | create(persona), createJudge()                          |

Таблица 25: Класова йерархия на агентите

### AgentOrchestrator (Chain of Responsibility + конкурентност)

- sessions: Map<string, DebateSession> - активни сесии, ключ debate id (ConcurrentHashMap-стил).
- ROUND\_PHASES: RoundType[] - редът на рундовете.
- runSession() - главният цикъл: обхожда рундовете, после агентите; for await (const token of agent.generateResponse(ctx)) стриймва асинхронно.
- roundGate: (() => void) | null - порта за step режим: блокира до advance().
- subscribe() връща Observable<DebateEvent> за SSE.

### LLM интеграция

OpenAIStreamingClient (agent/infrastructure/openai-streaming-client.ts) имплементира port-a ILLMClient. Модел по подразбиране gpt-4o-mini (от OPENAI\_DEFAULT\_MODEL). streamCompletion(messages, options): AsyncIterable<string> yield-ва токени при пристигане. Закачен в agent.module.ts под symbol LLM\_CLIENT.

### Обработка на PDF

PdfBillTextExtractor (port IBillTextExtractor): проверка на magic bytes (%PDF-), извличане с pdf-parse, детекция на сканиран PDF (твърде малко текст → ScannedPdfException), ограничения 200 - 200 000 символа, почистване на български маркери за страници (стр. 1 от 5).

### Custom exceptions

Всички наследяват ApplicationException (statusCode, code):

| Изключение                    | HTTP | code                      |
|-------------------------------|------|---------------------------|
| DebateNotFoundException       | 404  | DEBATE_NOT_FOUND          |
| PersonaGenerationException    | 422  | PERSONA_GENERATION_FAILED |
| BillParsingException          | 422  | BILL_PARSING_FAILED       |
| DebateAlreadyRunningException | 409  | -                         |
| DebateNotStartableException   | 400  | -                         |
| BillTooLongException          | 413  | BILL_TOO_LONG             |
| InvalidPdfException           | 400  | INVALID_PDF               |
| ScannedPdfException           | 422  | SCANNED_PDF               |

| Изключение              | HTTP | code            |
|-------------------------|------|-----------------|
| AnalysisFailedException | 422  | ANALYSIS_FAILED |

Таблица 26: Custom exceptions

### 3.3 Клиентска част - React

#### Маршрути

| Път                 | Страница           | Достъп   |
|---------------------|--------------------|----------|
| /                   | DashboardPage      | защитен  |
| /login              | LoginPage          | публичен |
| /register           | RegisterPage       | публичен |
| /profile            | ProfilePage        | защитен  |
| /debates/new        | CreateDebatePage   | защитен  |
| /debates/:id        | DebateRoomPage     | защитен  |
| /debates/:id/review | ReviewPersonasPage | защитен  |
| /synthesis/:id      | SynthesisPage      | защитен  |

Таблица 27: Клиентски маршрути

#### Feature модули

- **auth** - AuthProvider (Supabase сесия, sign-in/up/out), ProtectedRoute, EmailConfirmationCard.
- **debates** - ядрото: dashboard картички, стая на дебата, playback логика, API хукове, lib помощници.
- **synthesis** - verdict grid, shift секция, заключителна карта, PDF експорт (lazy @react-pdf).
- **profile** - заявки и мутации за профила.

#### SSE стрийминг

Хукът use-debate-stream.ts отваря EventSource на /debates/:id/stream докато дебатът тече. Парсва DebateEvent JSON и обновява състоянието по тип:

| Event           | Действие                                              |
|-----------------|-------------------------------------------------------|
| round_start     | задава текущия рунд, isStreaming=true                 |
| persona_start   | добавя нов StreamedMessage                            |
| token           | долепя токена към активната персона                   |
| persona_end     | маркира готова реплика, прихваща емоция               |
| round_end       | waitingForAdvance=true                                |
| debate_complete | isComplete=true, затваря потока, invalidate-ва заявки |

Таблица 28: SSE event-u

## Двата режима

- **Auto-play** - `use-playback.ts` (записани съобщения, авто-преход на 4s) и `use-live-playback.ts` (live курсор - авансира едва когато текущата персона е спряла да стриймва).
- **Стъпка-по-стъпка** - `useStartDebateMutation({ mode: "step" })`; бутон "Continue" се показва при `waitingForAdvance`, който вика `useAdvanceDebateMutation()` → POST `/debates/:id/advance`.

## TanStack Query

Заявки: `useDebatesQuery`, `useDebateDetailQuery`, `useDebateOverviewQuery`, `useDebateStatusQuery` (polling 2s докато Analyzing/Draft), `useChamberStatsQuery`, `usePersonasQuery`, `useSynthesisQuery`, `useProfileQuery`. Мутации: `create / update-personas / start / advance / delete debate`, `regenerate synthesis`, `update / delete profile`.

## Основни компоненти

- **Dashboard:** `ChamberHeader`, `ActiveNowPanel`, `FilterPills`, `DebateCard`, `DebateEmptyState`.
- **Стая (Stage view):** `StageView` (персони на кръгла сцена), `BillSummaryPanel`, `PersonaListPanel`, `ActiveQuoteCard`, `PlaybackControls`, `DebateFlowStepper`.
- **Стая (Chamber view):** `DebateTranscript` (история по рундове).
- **Синтез:** `VerdictGrid` (противоречия / общи точки / компромис), `ShiftSection` + `ShiftSparkline`, `JudgeClosingCard`.
- **Споделени:** `PersonaAvatar` (инициал, цвят, `emotion glow`), `PersonaStack`, `StatusPill`.

## 3.4 База данни - модели и заявки

Достъпът минава през Prisma зад domain repository port-ове (Repository pattern). `PrismaService` (глобален `@Global()` модул) разширява `PrismaClient` и управлява връзката (`$connect / $disconnect`).

Ключови шаблони:

- **Eager** на `persona` + `round.roundNumber` при четене на съобщения (избягва N+1 в чата).
- **Lazy** на пълната `persona` история - зарежда се отделно при нужда.
- **Каскадно изтриване** с `$transaction()` в обратен топологичен ред (`messages` → `rounds` → `analysis` → `judge` → `personas` → `debate`).
- **Индекси** за наредени заявки: (`round_id`, `sequence`), (`debate_id`, `created_at`), (`debate_id`, `round_number`).

Prisma client се билд-ва за `native + linux-musl-openssl-3.0.x (+ arm64)` за Alpine контейнерите.

## 3.5 Тестване

- **Unit тестове:** `AgentOrchestrator` (ред на рундовете, edge cases), `JudgeAgent`, `playback` редюсери (`use-playback.test.ts`, `use-live-playback.test.ts`), услуги.

- Покритие  $\geq 50\%$
- **Стартиране:** `pnpm test (turbo)`; пуска се и автоматично в pre-push hook и в CI.

## 4. Инфраструктура

### 4.1 Docker конфигурация

Отделни Dockerfile-и на база `node:22-alpine + pnpm`:

- **API** (`apps/api/Dockerfile`) - `pnpm install --filter @agora/api...`, билд на `@agora/shared`, `prisma generate`, `nest build`. `openssl` за Prisma, `EXPOSE 3000`, `CMD node apps/api/dist/main.js`.
- **Web** (`apps/web/Dockerfile`) - билд с baked `VITE_*` build args (`pnpm --filter @agora/web build`), после статиката се сервира с глобален `serve`. `EXPOSE 80`, `CMD serve -s apps/web/dist -l 80. -s (single-page mode)` дава SPA history fallback и връща 200 за `/health probe-a`.

`docker-compose.yml` за локален dev: `api (3000:3000) + web (5173:80, depends_on api)`.

### 4.2 CI/CD Pipeline

**CI** (`.github/workflows/ci.yml`) - на всеки PR към main и push към не-main клонове:

| Job          | Стъпки                                                                   |
|--------------|--------------------------------------------------------------------------|
| secrets-scan | gitleaks-action                                                          |
| lint         | install, db:generate, eslint, format:check, typecheck                    |
| test         | install, db:generate, pnpm test                                          |
| notify       | само при провал → Discord webhook (repo/branch/commit/actor/failed jobs) |

Таблица 29: CI pipeline

**CD** (`.github/workflows/cd.yml`) - на push към main и v\* тагове:

| Job                | Стъпки                                                                                                                                                                                            |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| build-and-push     | matrix (api, web): GHCR login, buildx (linux/amd64), push с тагове sha-short, sha-long, latest (main), semver (тагове). Build args: VITE_API_URL, VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY       |
| deploy (само main) | пренаписва image таговете в <code>k8s/api-deployment.yml</code> и <code>k8s/web-deployment.yml</code> на <code>:sha-&lt;short&gt;</code> (sed), commit-ва обратно в main с <code>[skip ci]</code> |
| notify             | при провал → Discord                                                                                                                                                                              |

Таблица 30: CD pipeline

Image-и в GHCR: `ghcr.io/<owner>/agora-api`, `ghcr.io/<owner>/agora-web`.

**GitOps доставка (ArgoCD).** Deploy-ът към клъстера не минава през ръчен `kubectl apply`. `argocd/application.yaml` дефинира ArgoCD Application, който следи `k8s/` на `main` и авто-синхронизира (`prune + selfHeal`) в namespace `agora`. Целият поток е автоматичен: `merge` в `main` → `CD build`-ва и `push`-ва `image`-ите → `deploy job`-ът пинва деплойментите на `:sha-<short>` и `commit`-ва обратно → ArgoCD синхронизира новите тагове в клъстера. Application-ът игнорира `secret.yaml` и `/data diff`-овете по `secret`-ите (`out-of-band`), както и `HPA status / spec.metrics drift`; не синхронизира `k8s/observability/`. Bootstrap веднъж: `kubectl apply -f argocd/application.yaml`.

#### Pre-commit hooks (Husky):

- `pre-commit - lint-staged (eslint --fix + prettier) → npm typecheck`.
- `pre-push - gitleaks (staged + нови комити) → npm test`.

**Secrets management:** GitHub Secrets (`GITLEAKS_LICENSE`, `DISCORD_WEBHOOK_URL`, `VITE_SUPABASE_ANON_KEY`) за CI/CD; `k3s` Secrets за `production` (`DATABASE_URL`, `SUPABASE_*`, `OPENAI_API_KEY`). `.gitleaks.toml` `allowlist`-ва `.env.example`, `README` и `docs`. `.env` е `gitignored`.

## 4.3 Kubernetes и наблюдаемост

Манифести в `[k8s/](../k8s/)` (IaC, всичко в `repo`). Оркестратор **k3s** с вграден Traefik. Доставката им в клъстера е GitOps през ArgoCD (`argocd/application.yaml`, виж 4.2).

| Файл                                                          | Какво декларира                                                                                          |
|---------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <code>namespace.yaml</code>                                   | namespace <code>agora</code>                                                                             |
| <code>secret.yaml</code>                                      | шаблон за secrets ( <code>DATABASE*URL</code> , <code>SUPABASE*\\</code> , <code>OPENAI_API_KEY</code> ) |
| <code>api-deployment.yaml</code>                              | 3 реплики, port 3000, initContainer (Prisma migrate), non-root 1001, лимити 500m CPU / 512Mi             |
| <code>web-deployment.yaml</code>                              | 2 реплики, serve port 80, лимити 200m CPU / 128Mi                                                        |
| <code>api-service.yaml</code> / <code>web-service.yaml</code> | ClusterIP services                                                                                       |
| <code>api-hpa.yaml</code>                                     | min 3 / max 6, CPU 70%                                                                                   |
| <code>web-hpa.yaml</code>                                     | min 2 / max 4, CPU 70%                                                                                   |
| <code>ingress.yaml</code>                                     | Traefik IngressRoute - /api (prio 100), / (prio 10)                                                      |

Таблица 31: Kubernetes манифести

#### Наблюдаемост (`k8s/observability/`):

- **Prometheus** (v2.52.0) - scrape 15s, цели `agora-api:3000/metrics`; alert rules: `HighHttpRequestRate` (>5% за 5m), `OpenAILatencyHigh` (p95 > 30s), `NoActiveSSEConnections` (0 за 30m).
- **Alertmanager** (v0.27.0) - receiver Discord webhook, group 10s / repeat 1h.
- **Grafana** (10.4.3) - dashboard "Agora Observability": Debates Created, Active SSE Connections, HTTP Error Rate, OpenAI Latency (p50/p95), Debate Generation Duration.

Метрики, изложени от API (MetricsService, prom-client):

| Метрика                                  | Тип       | Какво                                |
|------------------------------------------|-----------|--------------------------------------|
| agora_debates_created_total              | Counter   | създадени дебати (rate → дебати/час) |
| agora_http_requests_total                | Counter   | всички заявки                        |
| agora_http_errors_total                  | Counter   | 4xx/5xx, label status_code           |
| agora_openai_request_duration_seconds    | Histogram | латентност към OpenAI                |
| agora_sse_connections_active             | Gauge     | активни SSE връзки                   |
| agora_debate_generation_duration_seconds | Histogram | времетраене на цял дебат             |

Таблица 32: Prometheus метрики

4.4 Инструкции за стартиране

Изисквания: Node ≥ 20, pnpm ≥ 10, Docker (+ gitleaks за hooks: brew install gitleaks). База: Supabase (Postgres).

Стъпка 1 - конфигурация:

```
cp .env.example .env          # попълни DATABASE_URL + SUPABASE_* + OPENAI_API_KEY
```

Листинг 2: Конфигурация на средата

Стъпка 2A - Docker (production-подобно):

```
docker compose up --build      # web :5173, api :3000
```

Листинг 3: Стартиране с Docker

Стъпка 2B - dev с hot reload:

```
pnpm install
pnpm dev                        # web :5173, api :3001 (проксиран като /api)
```

Листинг 4: Dev режим с hot reload

Качество:

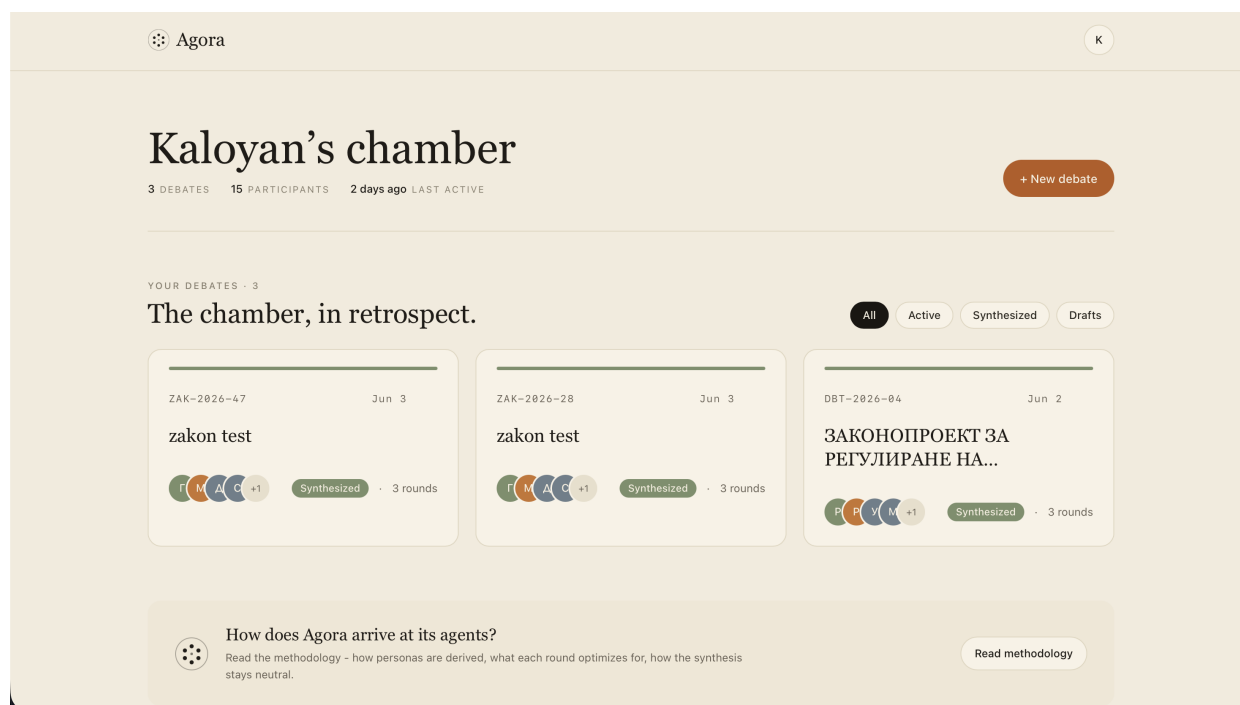
```
pnpm build | typecheck | test | lint
```

Листинг 5: Проверки за качество

5. Екранни снимки

Табло (Dashboard)





Фигура 4: Табло (Dashboard)

Личната "камара" на потребителя: обобщена статистика (брой дебати, участници, последна активност), филтри (All / Active / Synthesized / Drafts) и картички за всеки дебат - код, дата, стек от аватари на персоните, статус и брой рундове. Долу - вход към методологията на Agora.

## Стая на дебата (Stage view)

K

← Dashboard

ZAK-2026-47 · UPLOADED JUN 3

Stage view

Chamber view

Synthesized

# zakon test

DEBATE FLOW

1

Opening positions

Each agent states their initial stance.

2

Counter-arguments

Agents respond directly to others.

3

Common ground

Where compromise is possible.

4

Synthesis

Neutral judge writes a summary.

See the synthesis →

THE BILL, IN SHORT

\$1

въвеждане на нови подфондове

\$2

промени в условията за инвестиции

\$3

увеличаване на минималната капиталова база

\$4

регламентиране на нови видове пенсии

AT THE TABLE

Г. Георгиев

Възраст: 65+, Доход: нисък, Регион: цялата страна, Заетост: безработни

М. Петрова

Възраст: 30-60, Доход: висок, Регион: големи градове, Заетост: частен сектор

Д. Иванов

Възраст: 35-55, Доход: висок, Регион: големи градове, Заетост: финансов сектор

С. Николов

Възраст: 30-60, Доход: среден, Регион: столицата, Заетост: държавен сектор

Л. Василева

Възраст: 25-50, Доход: среден, Регион: големи градове, Заетост: частен сектор

Г

Г. Георгиев

Възраст: 65+, Доход: нисък, Регион: цялата страна, Заетост: безработни

ROUND 2

FEELING — ● CONFIDENT

“Аз не мога да се съглася с мнението на г-ца Петрова, която посочва, че увеличаването на минималната капиталова база ще натовари бизнеса и ще доведе до допълнителни разходи. Това е погрешно, защото новите изисквания за капиталова база всъщност ще осигурят по-голяма стабилност на пенсионните фондове. Увеличаването на минималната капиталова база е важно за защитата на пенсионерите и за гарантиране на техните права, а не за натоварване на бизнеса. Прехвърлянето на средства между фондовете, което тя оспорва, ще предостави важна гъвкавост на осигурените лица, позволявайки им да управляват активите си по най-добрия начин. Подкрепям законопроекта, защото той е насочен към укрепване на пенсионната система и осигуряване на дългосрочна защита на пенсионерите.”

←

▶

→

06 / 15

Skip to synthesis →

Фигура 5: Стая на дебата (Stage view)

Кръгла "сцена" с персонаите, разположени около централния код на законопроекта и текущия рунд. Ляво - stepper на потока (Opening positions → Counter-arguments → Common ground → Synthesis). Дясно - ключовите промени в законопроекта и списъкът с участниците ("At the table"). Долу - активната реплика, която се стриймва token-by-token, с playback контроли (play/pause, прев/след, проргес, skip to synthesis).

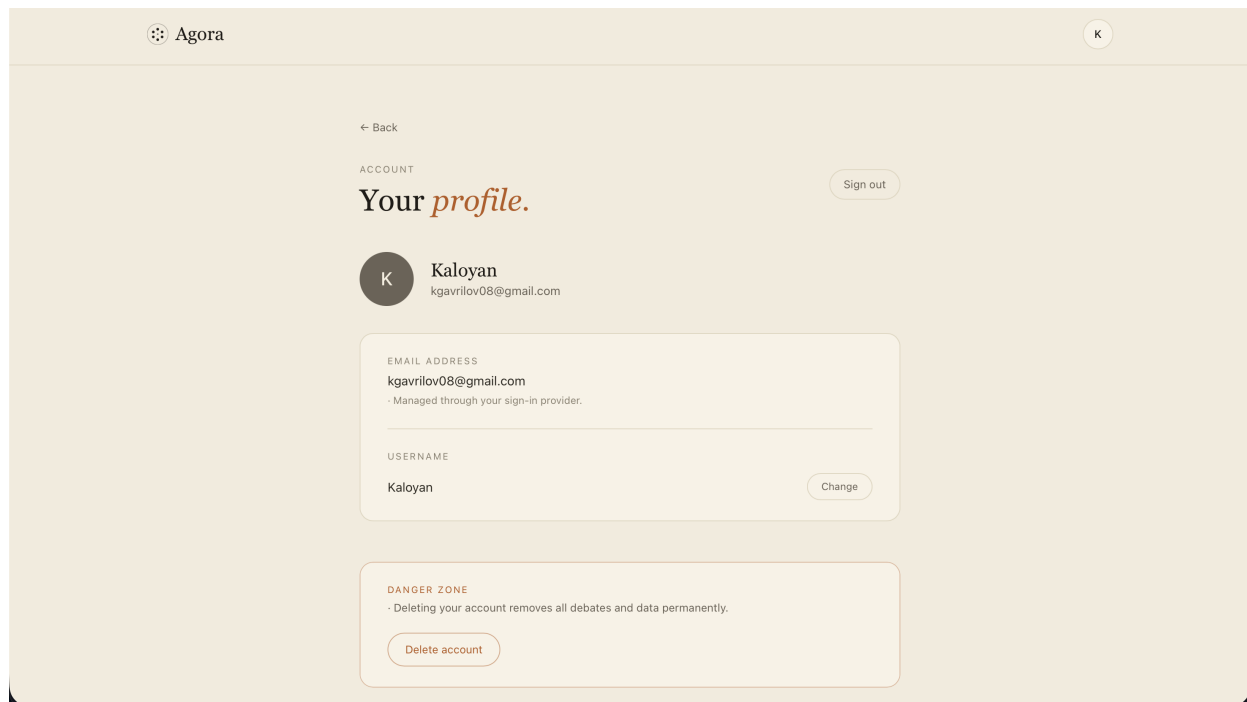
## Синтез (заключение)



Фигура 6: Синтез (заклучение)

Изводът на съдията: три колони - Contradictions / Common ground / Compromise. Секцията "How each seat shifted" показва за всяка персона как се е променила позицията ѝ (с процент на изместване). Долу - заключителното изявление на съдията и бутон за PDF експорт.

## Профил



Фигура 7: Профил

Профил на потребителя: email (управляван от sign-in доставчика), промяна на потребителско име и "Danger zone" за изтриване на акаунт с каскадно изтриване на всички данни.

## 6. AI инструменти

### 6.1 Claude Code

CLI инструмент за генериране и рефакториране на код, използван по време на разработката - писане на компоненти, услуги, тестове и тази документация.

### 6.2 GitHub Copilot

Подпомагане при писане на код в редактора и автоматизирани предложения при PR review.

## Заклучение

**Постигнати резултати.** Agora реализира пълен цикъл: качване на законопроект → автоматичен анализ → динамично генериране на персонализирани AI агенти → структуриран дебат в рундове с реалновремени стрийминг → неутрален синтез с конкретни компромисни точки. Системата стъпва на слоеста архитектура и от двете страни, ясно видими design patterns

(Strategy, Chain of Responsibility, Factory, Repository, DI), пълна наследствена йерархия BaseAgent → PersonaAgent | JudgeAgent | AnalysisAgent, нормализирана (3NF) PostgreSQL схема и production-grade инфраструктура (k3s, HPA, Traefik, Prometheus/Grafana/Alertmanager, CI/CD към GHCR).

**Научено.** Стрийминг архитектура с SSE и backpressure per-connection; оркестрация на множество асинхронни LLM генерации; следване на JSON схеми за структуриран изход от LLM; декларативно разгръщане в Kubernetes.

## Източници

1. Портал за обществени консултации - [strategy.bg](https://strategy.bg)
2. OpenAI API документация - [platform.openai.com/docs](https://platform.openai.com/docs)
3. NestJS документация - [docs.nestjs.com](https://docs.nestjs.com)
4. Prisma документация - [prisma.io/docs](https://prisma.io/docs)
5. Supabase документация - [supabase.com/docs](https://supabase.com/docs)
6. k3s / Traefik документация - [k3s.io](https://k3s.io), [traefik.io](https://traefik.io)
7. Prometheus / Grafana документация - [prometheus.io](https://prometheus.io), [grafana.com/docs](https://grafana.com/docs)

## Приложения

- GitHub репозитори: <https://github.com/TUES-2026-PBL-11-klas/Zlaten-Vek-Agora>